Splitting Method for Optimal Control

ENSIMAG - UGA
M1AM
Graduate School Project: Splitting Method for Optimal Control - Le Quoc Tung and Thomas Guilmeau

---

BOYER Timothé, HACINI Malik, LAINE Martin, MOURET Basile
18.04.2026 -

1 Introduction

Optimal control is the task of choosing inputs for a dynamical system so that it follows a desired behavior while minimizing a cost. In many applications, the system state (position, inventory, portfolio, etc.) evolves over time, and each decision affects both the current cost and future states. This temporal coupling is what makes optimal control both important and challenging.

In this project, we study the discrete-time finite-horizon setting. At each time step t, the state $x_t \in \mathbb{R}^n$ and control $u_t \in \mathbb{R}^m$ are linked by linear dynamics,

$$x_{t+1} = A_t x_t + B_t u_t + c_t, \tag{1}$$

with a fixed initial state $x_0 = x_{\text{init}}$. The objective is to minimize a sum of stage costs of the form $\varphi_t(x_t, u_t) + \psi_t(x_t, u_t)$ over the full horizon. In our setting, φ_t is quadratic and smooth, while ψ_t is convex and can encode nonsmooth penalties or constraints.

We focus on the splitting method of O'Donoghue, Stathopoulos, and Boyd [1] because it exploits the problem structure directly, which makes repeated solves and warm starts efficient. Alternative methods such as CVX, CVXGEN, and fast MPC can also be used for related optimal-control problems, but OSC is especially well suited to our setting. It separates the problem into two efficient steps: a structured quadratic step with linear dynamics and a stage-wise proximal step.

The goal of this work is to reproduce and study this method in Julia, using the original C implementation as a reference. We first present the mathematical formulation and the ADMM-based splitting updates. We then describe the main implementation choices: data structures, factorization caching, and in-place proximal updates. Finally, we evaluate the solver on the benchmark examples from the paper and compare iteration counts and runtimes with the C implementation.

2 Method

The Julia solver implemented in this project follows the operator-splitting method proposed by O'Donoghue, Stathopoulos, and Boyd [1] for finite-horizon linear-convex optimal control. The presentation below mirrors the paper's formulation and isolates the two computational primitives used in the implementation: a structured quadratic control solve and a stage-wise proximal update.

2.1 Problem Formulation

For a horizon T, let $x_t \in \mathbb{R}^n$ and $u_t \in \mathbb{R}^m$ denote the state and control at stage t. We consider the finite-horizon problem

$$\min_{x,u} \sum_{t=0}^{T} (\varphi_t(x_t, u_t) + \psi_t(x_t, u_t)) \tag{2}$$

subject to the dynamics $x_{t+1} = A_t x_t + B_t u_t + c_t$ for $t = 0, ..., T-1$, and $x_0 = x_{\text{init}}$.

The quadratic part of the stage cost is

$$\varphi_t(x, u) = \frac{1}{2} x^T Q_t x + x^T S_t u + \frac{1}{2} u^T R_t u + q_t^T x + r_t^T u, \tag{3}$$

with a positive-semidefinite block matrix $\begin{pmatrix} Q_t & S_t \\ S_t^T & R_t \end{pmatrix}$. The term ψ_t is assumed to be closed, proper, and convex; by allowing extended values, it can also encode convex constraints through indicator functions. Over the whole horizon, define

$$\varphi(x,u) = \sum_{t=0}^{T} \varphi_t(x_t,u_t), \quad \psi(x,u) = \sum_{t=0}^{T} \psi_t(x_t,u_t), \tag{4}$$

and let D be the affine set of trajectories satisfying the dynamics and the initial condition.

The optimal control problem can then be written compactly as

$$\min_{x,u} I_D(x,u) + \varphi(x,u) + \psi(x,u). \tag{5}$$

Where I_D is null if $(x,u) \in D$ and takes the value $+\infty$ otherwise, enforcing feasibility of the solution point. This decomposition keeps the quadratic part and the linear dynamics together, while separating the nonsmooth or constraint-encoding terms.

2.2 Consensus Splitting

To split these two parts algorithmically, we introduce auxiliary variables $(\tilde{x},\tilde{u})$ as a copy of the trajectory and write

$$\min_{x,u,\tilde{x},\tilde{u}} I_D(x,u) + \varphi(x,u) + \psi(\tilde{x},\tilde{u}) \tag{6}$$

subject to $(x,u) = (\tilde{x},\tilde{u})$.

The consensus constraint $(x,u) = (\tilde{x},\tilde{u})$ forces the original variables and their copy to coincide componentwise at the solution.

Applying scaled ADMM with penalty parameter $\rho > 0$ and scaled dual variables (z,y) gives the iteration

$$\begin{aligned}
\left(x^{k+1},u^{k+1}\right) &= \arg\min_{x,u} I_D(x,u) + \varphi(x,u) + \frac{\rho}{2}\left\|(x,u) - \left(\tilde{x}^k,\tilde{u}^k\right) - \left(z^k,y^k\right)\right\|_2^2 \\
\left(\tilde{x}^{k+1},\tilde{u}^{k+1}\right) &= \arg\min_{\tilde{x},\tilde{u}} \psi(\tilde{x},\tilde{u}) + \frac{\rho}{2}\left\|(\tilde{x},\tilde{u}) - \left(x^{k+1},u^{k+1}\right) + \left(z^k,y^k\right)\right\|_2^2 \\
\left(z^{k+1},y^{k+1}\right) &= \left(z^k,y^k\right) + \left(\tilde{x}^{k+1},\tilde{u}^{k+1}\right) - \left(x^{k+1},u^{k+1}\right).
\end{aligned} \tag{7}$$

The second update separates across time because $\psi(\tilde{x},\tilde{u}) = \sum_{t=0}^{T} \psi_t(\tilde{x}_t,\tilde{u}_t)$. Thus, for each stage t:

$$\left(\tilde{x}_t^{k+1},\tilde{u}_t^{k+1}\right) = \arg\min_{\bar{x},\bar{u}} \psi_t(\bar{x},\bar{u}) + \frac{\rho}{2}\left\|(\bar{x},\bar{u}) - \left(x_t^{k+1} - z_t^k, u_t^{k+1} - y_t^k\right)\right\|_2^2, \tag{8}$$

which is the proximal operator of $\frac{\psi_t}{\rho}$ evaluated at $\left(x_t^{k+1} - z_t^k, u_t^{k+1} - y_t^k\right)$.

The first update is a convex quadratic optimal control problem with linear dynamics.

This is the key advantage of the method: the only horizon-coupled step is quadratic, while the nonsmooth part reduces to independent small problems that can often be solved in closed form.

2.3 Quadratic Step

Define the stacked primal variable $w = (x_0,u_0,x_1,u_1,...,x_T,u_T)$.

For each stage t, define

$$E_t = \begin{pmatrix} Q_t + \rho I & S_t \\ S_t^T & R_t + \rho I \end{pmatrix}. \tag{9}$$

Since $\begin{pmatrix} Q_t & S_t \\ S_t^T & R_t \end{pmatrix}$ is positive semidefinite and $\rho > 0$, each matrix E_t is symmetric positive definite. Let E be the block diagonal matrix with diagonal blocks $E_0, ..., E_T$. Let h denote the right-hand side vector of the constraints and f the linear objective vector:

$$\begin{aligned} h &= (x_{\text{init}}, c_0, c_1, ..., c_{T-1}) \\ f &= \big(q_0 - \rho(\tilde{x}_0^k + z_0^k), r_0 - \rho(\tilde{u}_0^k + y_0^k), ..., q_T - \rho(\tilde{x}_T^k + z_T^k), r_T - \rho(\tilde{u}_T^k + y_T^k)\big) \end{aligned} \tag{10}$$

Finally, let G be the block-sparse linear operator such that $Gw = h$ is equivalent to the constraints in Eq. 2. With these definitions, the quadratic step in Eq. 7 is equivalent, after expanding the squared norm and dropping terms independent of w, to the quadratic program

$$\min_w \frac{1}{2} w^T E w + f^T w \tag{11}$$

subject to $Gw = h$.

The associated optimality conditions are the KKT system

$$\begin{pmatrix} E & G^T \\ G & 0 \end{pmatrix} \begin{pmatrix} w \\ \lambda \end{pmatrix} = \begin{pmatrix} -f \\ h \end{pmatrix}. \tag{12}$$

For fixed A_t, B_t, Q_t, S_t, R_t, and ρ, the matrix in Eq. 12 is constant across ADMM iterations. The implementation therefore factorizes this matrix once by a sparse LDL^T factorization and reuses the factorization throughout a solve, and also across warm-started solves whenever the structural data stay unchanged.

2.4 Convergence Criterion

Convergence is monitored through the primal and dual residuals associated with the consensus constraint Eq. 6:

$$r^k = \big(x^k, u^k\big) - \big(\tilde{x}^k, \tilde{u}^k\big), \quad s^k = \rho\big(\big(\tilde{x}^k, \tilde{u}^k\big) - \big(\tilde{x}^{k-1}, \tilde{u}^{k-1}\big)\big). \tag{13}$$

The algorithm stops when both residual norms are sufficiently small,

$$\big\|r^k\big\|_2 \le \varepsilon_{\text{pri}} \quad \text{and} \quad \big\|s^k\big\|_2 \le \varepsilon_{\text{dual}}, \tag{14}$$

with thresholds

$$\varepsilon_{\text{pri}} = \varepsilon_{\text{abs}}\sqrt{(T+1)(n+m)} + \varepsilon_{\text{rel}} \max\Big(\big\|\big(x^k, u^k\big)\big\|_2, \big\|\big(\tilde{x}^k, \tilde{u}^k\big)\big\|_2\Big), \tag{15}$$

$$\varepsilon_{\text{dual}} = \varepsilon_{\text{abs}}\sqrt{(T+1)(n+m)} + \varepsilon_{\text{rel}}\big\|\big(z^k, y^k\big)\big\|_2. \tag{16}$$

Following [1], one may also use relaxation by replacing $\big(x^{k+1}, u^{k+1}\big)$ in the proximal and dual updates with

$$\alpha\big(x^{k+1}, u^{k+1}\big) + (1-\alpha)\big(\tilde{x}^k, \tilde{u}^k\big), \quad \alpha \in (0, 2), \tag{17}$$

which often improves empirical convergence.

3 Implementation

The implementation aims to provide a high-performance solver through a high-level public interface. Julia is a natural choice for this purpose, as it combines efficient LLVM-based compilation with a dynamic type system. This makes it well suited to optimization algorithms, which require both a high level of mathematical abstraction and strong performance. For this project, we used the mature **`LinearAlgebra`** library, which enables fast implementations of vector and matrix operations.

In order to keep the project clean, we separated the repository into multiple folders. The main package **`OptimalControl_OperatorSplitting`** is contained in the **`src/`** folder. It is split into multiple parts. In **`types.jl`** we defined the problem, iterate, cache, and timing structures. The **`utils.jl`** file is used for helper functions (constructors, sparse KKT assembly) as well as the convergence metrics. The main solver logic is then defined in **`solver.jl`** following the iteration in Eq. 7: it builds the right-hand side of the quadratic step, solves the linear system, applies the optional relaxation, evaluates the stage-wise proximal update Eq. 8, performs the dual update, and checks convergence with the residuals Eq. 13 and the stopping rule Eq. 14. Finally **`cache.jl`** assembles the KKT system Eq. 12 once and computes the sparse LDL^T factorization reused by the solver. This results in a simple public API, as the user only has to initialise the cache, define the proximal step, and pass it to the solver. For example, for a box-constrained problem:

```
function prox!(x_tilde, u_tilde, v, w, rho)
  x_tilde .= v
  u_tilde .= clamp.(w, umin, umax)
end

data = all_data(A, B, c, Q, S, R, q, r, x_init; rho=50.0, alpha=1.8)
cache = setup_cache(data)
x, u, tt = solve(cache, prox!; max_iters=3000)
```

We added some tests comparing the solver against an interior point method **`Ipopt.jl`** and checked that cache reuse and warm starts remain correct when the linear terms change. We also implemented the exact examples from the paper in order to compare performance with the **`C`** implementation.

Because Julia uses a garbage collector to handle memory management, reducing allocations is crucial for performance. We preallocate the ADMM variables and workspaces as vectors, use in-place proximal operators, and rely on the cache to reuse the factorization across iterations and warm-started solves.

4 Results

4.1 Performance comparison

To evaluate our implementation, we used the same benchmark problems, problem parameters, and convergence criterion as the reference paper. With this setup, we obtain exactly the same number of iterations to convergence as the original C implementation, for the same solutions, on every problem variant reported in the paper. This is a strong validation that the Julia solver faithfully reproduces the reference algorithm.

In terms of runtime, the Julia implementation remains in the same overall order of magnitude as the C code.

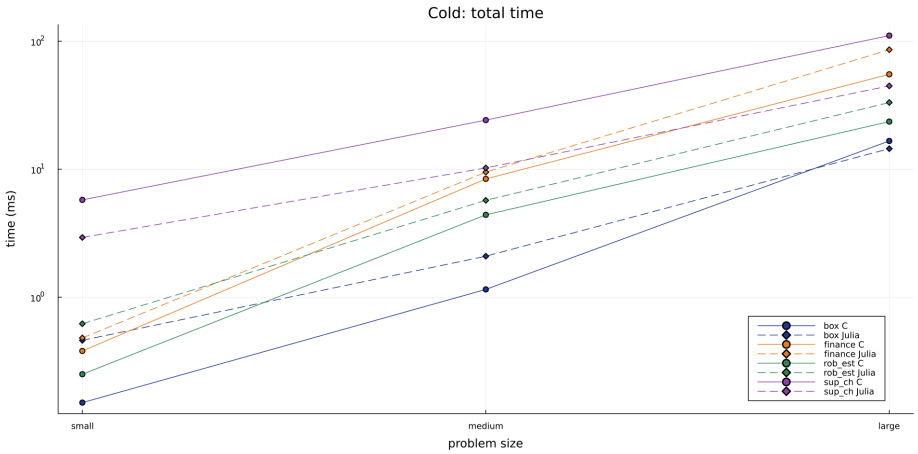

Figure 1 - Execution time of the algorithm for the different problems in the cold-start case.

Each iteration is split into two main steps: the linear-system solve and the proximal update. We therefore measured them separately to identify the source of the runtime differences.

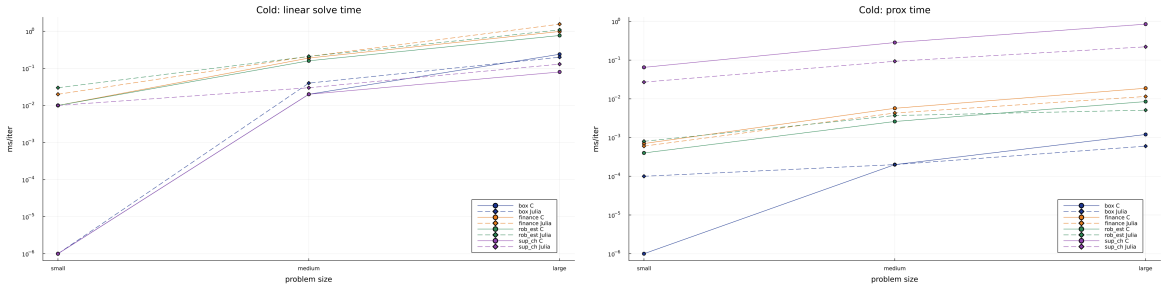

Figure 2 - Execution time of the linear solve (left) and the proximal update (right) for the different problems in the cold-start case.

We can see that Julia is slower on the linear solve but faster on the proximal step. The slower linear solve is mainly explained by memory allocations, which are more costly in Julia than in the reference C code.

The exact problem structure matters: when the proximal step is relatively expensive compared with the linear solve, Julia can be faster overall, as in the supply-chain example; on less favorable problems, it is slightly slower. Overall, the goal of high performance is achieved even relative to this low-level C implementation.

5 Conclusion

In this project, we implemented the operator-splitting method of O'Donoghue, Stathopoulos, and Boyd [1] in Julia for finite-horizon optimal control with linear dynamics. On the benchmark examples, the Julia solver matches the reference C implementation in iteration counts, while runtime differences depend on the relative costs of the linear-system and proximal steps.

Several extensions would be natural. We could study the sensitivity to the hyperparameters (ρ, α, and the stopping tolerances), broaden the comparison with other optimization methods, and test the solver on additional examples. A more complete theoretical study, including convergence-rate results, would also strengthen the analysis.

References

[1] B. O'Donoghue, G. Stathopoulos, and S. Boyd, "A Splitting Method for Optimal Control," *IEEE Transactions on Control Systems Technology*, vol. 21, no. 6, pp. 2432–2442, 2013, doi: 10.1109/TCST.2012.2231960.